

Dockerfile Avanzado

Arturo Silvelo

Try New Roads

Almacenamiento

Docker soporta los siguientes tipos de montajes de almacenamiento para guardar datos:

- **Volume mounts**
- **Bind mounts**
- **tmpfs mounts**

Volume mounts

Volume mounts

Los **volúmenes** son almacenes de datos persistentes para contenedores, creados y gestionados por Docker. Puedes crear un volumen explícitamente con el comando: `docker volume create`, o Docker puede crearlo durante la creación de un contenedor o servicio.

Gestión

Cuando se crea un **volumen** se almacena en un directorio del **host Docker**. Cuando se monta un volumen en un contenedor, ese directorio es lo que se monta dentro del contenedor.

Esto es similar a como funcionan los **bind mounts**, excepto que los volúmenes son gestionados por Docker y están aislados de la funcionalidad principal del sistema host.

Ubicación de almacenamiento:

- **Linux:** `/var/lib/docker/volumes/<volume-name>/_data`
- **Windows (Docker Desktop):** `\\wsl$\\docker-desktop-data\\data\\docker\\volumes\\<volume-name>\\_data`

- Opción 1

```
services:
  app-base:
    build:
      context: ../../../../app
      dockerfile: ../02-compose/ejemplos/Dockerfile
    ports:
      - "3001:3000"
    image: app-base
    container_name: app-base
    environment:
      NODE_ENV: development
      PORT: 3000
    volumes:
      - app-uploads-compose:/app/uploads

volumes:
  app-uploads-compose:
    driver: local
    name: app-uploads-compose
```

```
docker compose -f 03-volumes/ejemplos/1.volume/compose.yml up -d --build
```

- Opción 2

```
docker volume create app-uploads
docker run -p3000:3000 --name app-base -v app-uploads:/app/uploads app-base
```

- Verificación

```
docker volume ls
ls /var/lib/docker/volumes/
```

```
curl -X POST http://localhost:3001/upload \
-F "file=@./03-volumes/ejemplos/lenna.png" \
-H "Content-Type: multipart/form-data"
```

- Resultado

```
docker exec app-base-compose ls /app/uploads
docker exec app-base ls /app/uploads
```

```
ls /var/lib/docker/volumes/app-uploads/_data
ls /var/lib/docker/volumes/app-uploads-compose/_data
```

- Limpiar

```
docker rm -f app-base app-base-compose
docker volume rm app-uploads-compose app-uploads
```

Ciclo de vida

- Un volume dado puede ser montado en **múltiples contenedores simultáneamente**
- Cuando ningún contenedor está usando un volume, el volume sigue **disponible para Docker**
- Puedes eliminar volumes no utilizados usando: `docker volume prune`

Ventajas

- Se pueden gestionar usando comandos CLI de Docker o la API
- Funcionan tanto en contenedores Linux como Windows
- Se pueden compartir más seguramente entre múltiples contenedores
- Cuando tu aplicación requiere I/O de alto rendimiento
- Fáciles de migrar y hacer backup
- Se necesita tener contenido previo en el contenedor.

Named y Anonymous volumes

Un volumen puede ser **nombrado** o **anónimo**. Los volúmenes anónimos reciben un nombre aleatorio que está garantizado de ser único dentro de un host Docker dado.

Al igual que los volúmenes nombrados, **los volúmenes anónimos persisten** incluso si eliminas el contenedor que los usa, **excepto** si usas el flag `--rm` al crear el contenedor, en cuyo caso el volumen anónimo asociado con el contenedor es destruido.

Características de Anonymous volumes

Si creas múltiples contenedores consecutivamente que cada uno usa volúmenes anónimos, **cada contenedor crea su propio volumen**.

Los volúmenes anónimos **no son reutilizados o compartidos** entre contenedores automáticamente.

- compose.yml

```
x-app-base: &app-base
  build:
    context: ../../../../app
    dockerfile: ../03-volumes/ejemplos/Dockerfile.anonymous
  ports:
    - "3000:3000"
  image: app-base
  container_name: app-base
  environment:
    NODE_ENV: development
    PORT: 3000

services:
  app-base-1:
    <<: *app-base
  app-base-2:
    <<: *app-base
    container_name: app-base-2
    ports:
      - "3002:3000"
  app-base-3:
    <<: *app-base
    container_name: app-base-3
    ports:
      - "3003:3000"
```

- Dockerfile

```
VOLUME /app/uploads
VOLUME /app/logs
```

- Ejecución

```
docker compose -f 03-volumes/ejemplos/2.anonymous/compose.yml up -d --build
```

- Verificación

```
docker volume ls
DRIVER      VOLUME NAME
local       4c6d0a83e7cc576945aa7eca1a931af58a6568b7a9e4145d3148742d57d16390
local       4c9f5b83ced73c1a8974be98b9b217be1aa4ae31cc2601c8e5834ee898fe9b0d
local       603d9377cd6a6e74cdb0d77b927c03b9775f3e01e0beff86449bd477f1de8e51
local       a4a41d61068c9e18ac0ef0e4a20fbefb32ef158085b5f55ac933cab69a76e151
local       b24b0bd309a53b5d6c9dd03270390e51b650524818146cd121dbc921f2732578
local       e5c6355e0cfd95a064cf05be083960b067e95a7b894390bffa1ccaa064baba6ce
```

- Limpiar

```
docker compose -f 03-volumes/ejemplos/2.anonymous/compose.yml down -v
# ó
docker compose -f 03-volumes/ejemplos/2.anonymous/compose.yml down
docker volume prune
```

```
docker volume ls
DRIVER      VOLUME NAME
```

Montaje de volumes: --mount vs --volume

Para montar un volumen con el comando `docker run`, puedes usar los flags `--mount` o `--volume`.

La principal diferencia es que el flag `--mount` es más explícita y soporta todas las opciones disponibles.

Debes usar `--mount` si quieres:

- Especificar opciones del driver de volumen
- Montar un subdirectorio de un volumen
- Montar un volumen en un servicio de Swarm

Comparativa de opciones: --mount vs --volume

Opción	mount	volume	Descripción
source, src	✓	✗	La fuente del montaje. Para volúmenes nombrados, es el nombre del volumen.
destination, dst, target	✓	✗	La ruta donde el archivo o directorio se monta en el contenedor.
volume-subpath	✓	✗	Una ruta a un subdirectorio dentro del volumen para montar en el contenedor.
readonly, ro	✓	✓	Si está presente, hace que el volumen se monte como solo lectura.
volume-nocopy	✓	✓	Si está presente, los datos en el destino no se copian al volumen si está vacío.
volume-opt	✓	✗	Puede especificarse más de una vez, toma un par clave-valor del nombre y valor.

```
docker run -d --mount source=app-uploads-mount,target=/app/logs,ro --name app-base-mount app-base
docker run -d -v app-uploads-volume:/app/logs:ro --name app-base-volume app-base
```

```
docker volume ls
DRIVER      VOLUME NAME
local      86d082cdbabb590be11db783dde3be9f039d38ba10c1df587a725323a68eeca5
local      983d94bf98ca23ec46acfccbc00543edc9e6ec5df71eac6d86021daa9f1f1149
local      app-uploads-mount
local      app-uploads-volume
```

```
docker rm -f --volumes app-base-mount app-base-volume
docker volume rm app-uploads-volume app-uploads-mount
```

Montar un subdirectorio

- Creación y configuración del volumen

```
docker volume create logs
docker run --rm \
  --mount src=logs,dst=/logs \
  alpine mkdir -p /logs/app1 /logs/app2
```

- Crear contenedores

```
docker run -d --mount source=logs,target=/app/logs,volume-subpath=app1 --name app-base-1 app-base
docker run -d --mount source=logs,target=/app/logs,volume-subpath=app2 --name app-base-2 app-base
```

- Verificar

```
docker volume ls
DRIVER      VOLUME NAME
local      2ddcf36b0b174340a774fdc991d3c52c4d20d8d37ed8ded7f0038214990501ed
local      c4e4afc0cc8c8be3832893e24d5872483178fd8003a1942638517537b15a796c
local      logs
```

- Verificar

```
docker run --rm --mount src=logs,dst=/logs alpine cat /logs/app1/app.log
docker run --rm --mount src=logs,dst=/logs alpine cat /logs/app2/app.log
```

- Limpiar

```
docker rm -f --volumes app-base-1 app-base-2
docker volume rm logs
```

```
x-app-base: &app-base
  build:
    context: ../../../../app
    dockerfile: ../03-volumes/ejemplos/Dockerfile.anonymous
  ports:
    - "3000:3000"
  image: app-base
  container_name: app-base
  environment:
    NODE_ENV: development
    PORT: 3000
  depends_on:
    - init-logs
x-alpine: &alpine
  image: alpine
  volumes:
    - app-logs:/logs
  command: |
    sh -c "
      if [ ! -d '/logs/app1' ]; then
        mkdir -p /logs/app1 /logs/app2
        echo 'Directorios creados'
      else
        echo 'Directorios ya existen'
      fi
    "
  restart: "no"

services:
  app-base-1:
    <<: *app-base
    container_name: app-base-1
    volumes:
      - type: volume
        source: app-logs
        target: /app/logs
        volume:
          subpath: app1
  app-base-2:
    <<: *app-base
    container_name: app-base-2
    ports:
      - "3001:3000"
    volumes:
      - type: volume
        source: app-logs
        target: /app/logs
        volume:
          subpath: app2
  init-logs:
    <<: *alpine
  test:
    <<: *alpine
    command: |
      sh -c "
        ls -l /logs/app1
        cat /logs/app1/app.log
        ls -l /logs/app2
        cat /logs/app2/app.log
      "
    depends_on:
      - app-base-1
      - app-base-2
  volumes:
    app-logs:
      driver: local
      name: app-logs
```

- Ejecución

```
docker compose -f 03-volumes/ejemplos/3.mount/compose.yml up -d --build
```

- Validación

```
docker compose -f 03-volumes/ejemplos/3.mount/compose.yml logs test
```

- Limpiar

```
docker compose -f 03-volumes/ejemplos/3.mount/compose.yml down -v
docker volume rm app-logs
```

Compartir datos entre máquinas

Al construir aplicaciones tolerantes a fallos, puedes necesitar configurar múltiples réplicas del mismo servicio para que tengan acceso a los mismos archivos.

- Añadir lógica a tu aplicación para almacenar archivos en un sistema de almacenamiento de objetos en la nube como **Amazon S3**.
- Crear volúmenes con un **driver** que soporte escribir archivos a un sistema de almacenamiento externo como **NFS** o **Amazon S3**.

- compose.yaml

```
x-app-base: &app-base
  build:
    context: ../../../../app
    dockerfile: ../03-volumes/ejemplos/Dockerfile.anonymous
  ports:
    - "3000:3000"
  image: app-base
  container_name: app-base
  environment:
    NODE_ENV: development
    PORT: 3000

services:
  app-base-1:
    <<: *app-base
    container_name: app-base-1
    volumes:
      - nfs-uploads:/app/uploads

  app-base-2:
    <<: *app-base
    container_name: app-base-2
    ports:
      - "3001:3000"
    volumes:
      - nfs-uploads:/app/uploads

volumes:
  nfs-uploads:
    driver: local
    driver_opts:
      type: nfs
      o: addr=127.0.0.1,rw,nfsvers=4,async,nosuid
      device: ":/"
```

- NFS Server

```
$docker compose -f 03-volumes/ejemplos/6.shared/compose-nfs.yml up -d
$mount -vv -o vers=4,async,nosuid,rw, 127.0.0.1:/ /mnt
```

- Ejecución

```
docker compose -f 03-volumes/ejemplos/6.shared/compose.yml up -d
```

- Verificación

```
curl -X POST http://localhost:3000/upload -F "file=@<file-path>" -H "Content-Type: multipart/form-data"
```

```
$docker exec app-base-1 ls -la uploads
$docker exec app-base-2 ls -la uploads
$ls /mnt
```

- Limpiar

```
$docker compose -f 03-volumes/ejemplos/7.nfs/compose.yml down
$docker compose -f 03-volumes/ejemplos/7.nfs/compose-nfs.yml down
$sudo umount /mnt
```

Backup, restauración y migración

Los volúmenes son útiles para **backups**, **restauraciones** y **migraciones**. Usando el flag `--volumes-from` podemos crear un nuevo contenedor que monte los volúmenes del contenedor origen.

Backup

- Crear un nuevo contenedor

```
docker run -d -p3000:3000 --name app-base app-base
```

- Subir contenido

```
curl -X POST http://localhost:3000/upload -F "file=@<file-path>" -H "Content-Type: multipart/form-data"
```

- Verificación

```
docker exec app-base ls /app/uploads  
lenna_1759404432021.png  
lenna_1759404434278.png  
lenna_1759404435833.png
```

- Verificación

```
curl -s http://localhost:3001/upload | jq '.[].filename'  
"lenna_1759404432021.png"  
"lenna_1759404434278.png"  
"lenna_1759404435833.png"
```

- Montamos los volumes de `app-base` en el contenedor y generamos un `.tar`

```
docker run --rm --volumes-from app-base -v $(pwd):/backup alpine tar cvf /backup/uploads.tar /app/uploads
```

Restaurar

- Crear un nuevo contenedor

```
docker run -d -p3001:3000 --name app-base-restore app-base
```

- Verificación

```
docker exec app-base-restore ls /app/uploads
```

- Restaurar

```
docker run --rm --volumes-from app-base-restore -v $(pwd):/backup alpine tar xvf /backup/uploads.tar -C /app/uploads --strip 2
```

- Verificación

```
docker exec app-base-restore ls /app/uploads  
lenna_1759404432021.png  
lenna_1759404434278.png  
lenna_1759404435833.png
```

```
curl -s http://localhost:3001/upload | jq '.[].filename'  
"lenna_1759404432021.png"  
"lenna_1759404434278.png"  
"lenna_1759404435833.png"
```

- Limpiar

```
docker rm -f app-base app-base-restore --volumes  
docker volume prune
```

Bind Mounts

Bind Mounts

Cuando usas un **bind mount**, un archivo o directorio en la máquina host se monta desde el host hacia un contenedor. En contraste, cuando usas un **volume**, se crea un nuevo directorio dentro del directorio de almacenamiento de Docker en la máquina host, y Docker gestiona el contenido de ese directorio.

Cuándo usar bind mounts

- **Compartir código fuente** o artefactos de build entre un entorno de desarrollo en el host Docker y un contenedor.
- Cuando quieres **crear o generar archivos** en un contenedor y persistir los archivos en el filesystem del host.
- **Compartir archivos de configuración** desde la máquina host a los contenedores.

Consideraciones y limitaciones

- Los bind mounts tienen **acceso de escritura** a archivos en el host por defecto
- Los bind mounts se crean hacia el **Docker daemon host**, no el cliente
- Los contenedores con bind mounts están **fuertemente ligados al host**

Montaje de volumes: --mount vs --volume

Para montar un volumen con el comando `docker run`, puedes usar los flags `--mount` o `--volume`.

La principal diferencia es que el flag `--mount` es más explícita y soporta todas las opciones disponibles.

Diferencias en comportamiento:

- `--volume`: **automáticamente crea el directorio** en el host.
- `--mount` **NO crea automáticamente** un directorio.

Comparativa de opciones: --mount vs --volume

Opción	-- mount	-- volume	Descripción
source, src	✓	✗	La ubicación del archivo o directorio en el host. Puede ser una ruta absoluta o relativa.
destination, dst, target	✓	✗	La ruta donde el archivo o directorio se monta en el contenedor. Debe ser una ruta absoluta.
readonly, ro	✓	✓	Si está presente, hace que el bind mount se monte en el contenedor como solo lectura.
bind-propagation	✓	✓	Si está presente, cambia la propagación del bind (shared,slave,private,rshared,rslave,rprivate)

- Ejecución

```
docker run -d -v ./logs_volume:/app/logs --name app-base-volume app-base-volume
```

- **Problema:** Se crea el directorio `bind` pero con permisos de root (cli docker)

```
$ls -ld logs_volume/
drwxr-xr-x 2 root root 4096 oct  6 11:13 logs_volume/
```

- **Solución:** Crear el directorio previamente. Pero esto solo será válido si el usuario del contenedor y el host comparten el mismo `id`

```
mkdir logs_volume
```

```
$docker run --rm app-base-volume id
uid=1001(nodeuser) gid=1001(nodejs) groups=1001(nodejs),1001(nodejs)
$id
uid=1000(silvelo) gid=1000(silvelo) groups=1000(silvelo)
```

- **Solución:** Cambiar permisos de la carpeta al usuario del contenedor

```
chown 1001:1001 logs_volume
```

- **Solución:** Establecer los permisos a `otros`

```
chmod 777 logs_volume
```

- **Solución:** Usar el mismo id en ambos entorno

```
$docker exec app-base-volume cat /etc/passwd
node:x:1000:1000:./home/node:/bin/sh
nodeuser:x:1001:1001:./home/nodeuser:/sbin/nologin
```

```
USER node
```

- Ejecución

```
docker run -d --mount type=bind,src=./logs_volume,dst=/app/logs --name app-base-volume app-base-volume
```

- compose.yaml

```
x-app-base: &app-base
  build:
    context: ../../../../app
    dockerfile: ../03-volumes/ejemplos/Dockerfile
  ports:
    - "3000:3000"
  image: app-base
  container_name: app-base
  environment:
    NODE_ENV: development
    PORT: 3000

services:
  app-base-1:
    <<: *app-base
    container_name: app-base-1
    volumes:
      - type: bind
        source: ./logs_volume/
        target: /app/logs
```

- Ejecución

```
docker compose -f 03-volumes/ejemplos/4.bind/compose.yml up -d
```

tmpfs

tmpfs mounts

Un **tmpfs mount** es **temporal**, y solo se persiste en la memoria del host. Cuando el contenedor se detiene, el tmpfs mount se elimina, y los archivos escritos allí no se persisten.

Limitaciones

- **No puedes compartir tmpfs mounts entre contenedores**
- Esta funcionalidad **solo está disponible en Linux**
- Establecer permisos reseteen después del reinicio del contenedor

- Ejecución

```
docker run -d --name app-base-mount --mount type=tmpfs,destination=/app/logs app-base
```

```
docker run -d --mount type=tmpfs,destination=/app/logs --name app-base-volume app-base
```

- **Problema:** El montaje se crea con permisos de `root`

```
$docker exec app-base-volume ls -ld logs
drwxr-xr-x  2 root    root          4096 Oct  6 11:16 logs
```

- **Solución:** Establecer permisos o el usuario del montaje

```
docker run -d --name app-base-mount --mount type=tmpfs,destination=/app/logs,tmpfs-mode=0777 app-base
```

```
docker run -d --name app-base-volume --tmpfs /app/logs:uid=1001,gid=1001,mode=0755 app-base
```

- compose.yaml

```
x-app-base: &app-base
  build:
    context: ../../../../app
    dockerfile: ../03-volumes/ejemplos/Dockerfile
  ports:
    - "3000:3000"
  image: app-base
  container_name: app-base
  environment:
    NODE_ENV: development
    PORT: 3000

services:
  app-base-1:
    <<: *app-base
    container_name: app-base-volume
    volumes:
      - type: tmpfs
        target: /app/logs
        tmpfs:
          size: 100m
          mode: 0777

  app-base-tmpfs:
    <<: *app-base
    container_name: app-base-mount
    ports:
      - "3001:3000"
    tmpfs:
      - /app/logs:uid=1001,gid=1001,mode=0755
```

- Ejecución

```
docker compose -f 03-volumes/ejemplos/5.tmpfs/compose.yml up -d
```